



PANDUAN BELAJAR UML - SIM4LON

Untuk Persiapan Sidang Seminar Kerja Praktek

Tujuan: Memahami semua diagram UML yang ada di SIM4LON agar siap menjawab pertanyaan dosen penguji dengan percaya diri!



BAGIAN 1: PENGENALAN UML

Apa itu UML?

UML (Unified Modeling Language) adalah bahasa standar untuk memodelkan sistem perangkat lunak. Bayangkan seperti "blueprint" untuk membangun rumah - sebelum tukang membangun, arsitek harus buat gambar dulu.

Kenapa Pakai UML di SIM4LON?

Alasan	Penjelasan
Dokumentasi	Orang lain bisa pahami sistem tanpa baca semua kode
Komunikasi	Tim developer, client, dan dosen bisa diskusi pakai "bahasa" yang sama
Perencanaan	Sebelum coding, rancang dulu biar tidak bingung
Validasi	Cek apakah desain sudah sesuai kebutuhan user



BAGIAN 2: USE CASE DIAGRAM

Fungsi Use Case Diagram

Use Case Diagram menggambarkan **SIAPA** melakukan **APA** di sistem. Ini diagram paling "high-level" - tidak detail teknis, fokus ke fitur dari sudut pandang user.

Simbol-Simbol Use Case Diagram

1. AKTOR (Stick Figure / Orang-orangan)

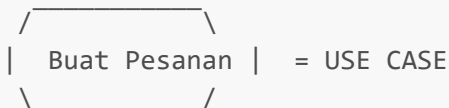
 = AKTOR

Arti: Orang atau sistem external yang berinteraksi dengan sistem kita.

Di SIM4LON ada 3 aktor:

Aktor	Peran	Contoh Aksi
 Admin	Pengelola sistem	Kelola user, lihat semua data
 Operator	Staff operasional	Input pesanan, update status
 Pangkalan	Pemilik pangkalan	Input penjualan, lihat stok sendiri

2. USE CASE (Oval / Elips)



Arti: Fitur atau aksi yang bisa dilakukan user.

Contoh Use Case di SIM4LON:

- Buat Pesanan
- Update Status
- Catat Pembayaran
- Lihat Laporan
- Voice Order

3. GARIS ASOSIASI (Garis Lurus)

Aktor — Use Case

Arti: Aktor ini bisa melakukan use case ini.

4. <> (Garis Putus-putus dengan Include)

Use Case A - - - -><<include>>- - - -> Use Case B

Arti: Use Case A **SELALU** membutuhkan Use Case B. Use Case B adalah bagian wajib dari A.

Contoh di SIM4LON:

Login - - - -><<include>>- - - -> Validasi Password

Artinya: Setiap login **PASTI** harus validasi password.

5. <> (Garis Putus-putus dengan Extend)

Use Case A <- - - -<<extend>>- - - Use Case B

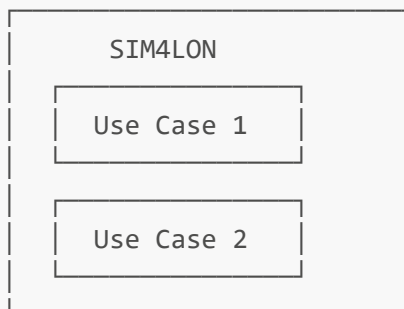
Arti: Use Case B adalah fitur **OPSIONAL** yang bisa menambah Use Case A.

Contoh di SIM4LON:

Buat Pesanan <- - - -<<extend>>- - - Voice Order AI

Artinya: Voice Order adalah cara alternatif untuk Buat Pesanan, tidak wajib.

6. System Boundary (Kotak Besar)



Arti: Batas sistem kita. Yang di dalam = tanggung jawab kita, yang di luar = external.

? Pertanyaan yang Mungkin Ditanya Dosen (Use Case)

Q: Kenapa ada 3 aktor berbeda?

A: Karena ada **Role-Based Access Control (RBAC)**. Setiap role punya hak akses berbeda. Admin bisa semua, Operator cuma operasional, Pangkalan cuma lihat data sendiri (multi-tenant).

Q: Apa bedanya include dan extend?

A: Include = **WAJIB** selalu dijalankan. Extend = **OPSIONAL** bisa dijalankan atau tidak.

Q: Kenapa Voice Order pakai extend?

A: Karena Voice Order adalah **cara alternatif** untuk input pesanan. User bisa pilih pakai voice atau manual - tidak wajib.



BAGIAN 3: ACTIVITY DIAGRAM

Fungsi Activity Diagram

Activity Diagram menggambarkan **ALUR PROSES** langkah demi langkah. Mirip flowchart, tapi lebih canggih karena bisa paralel dan ada swimlane.

Simbol-Simbol Activity Diagram

1. START NODE (Bulatan Hitam Penuh)

● = START (Mulai)

Arti: Titik awal proses. Setiap diagram harus punya 1 start.

2. END NODE (Bulatan Hitam dengan Lingkaran Luar)

⦿ = END (Selesai)

Arti: Titik akhir proses. Bisa ada lebih dari 1 (misal: sukses dan gagal).

3. ACTION / ACTIVITY (Kotak dengan Sudut Lengkung)

Buka Halaman = ACTION

Arti: Langkah/aksi yang dilakukan. Biasanya pakai kata kerja.

4. DECISION NODE (Diamond / Belah Ketupat)

◇
/ \
[Ya] [Tidak]

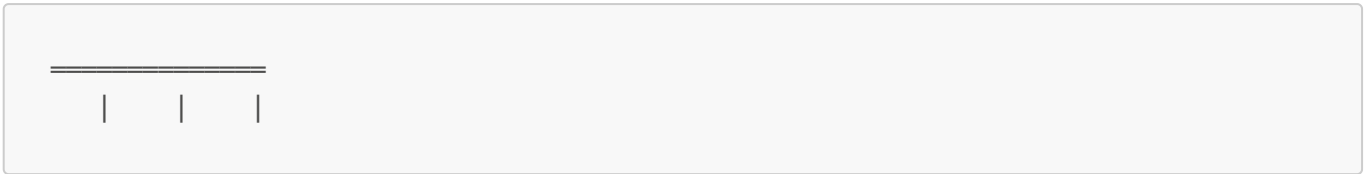
Arti: Percabangan berdasarkan kondisi. Harus ada minimal 2 jalur keluar.

5. MERGE NODE (Diamond tanpa Label)

◇
|

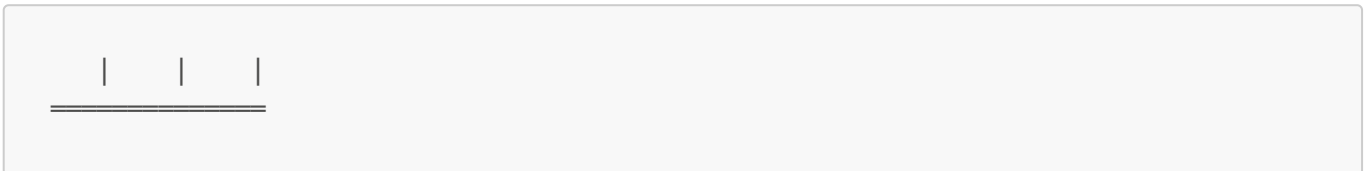
Arti: Menggabungkan kembali jalur yang terpecah dari decision.

6. FORK NODE (Garis Tebal Horizontal)



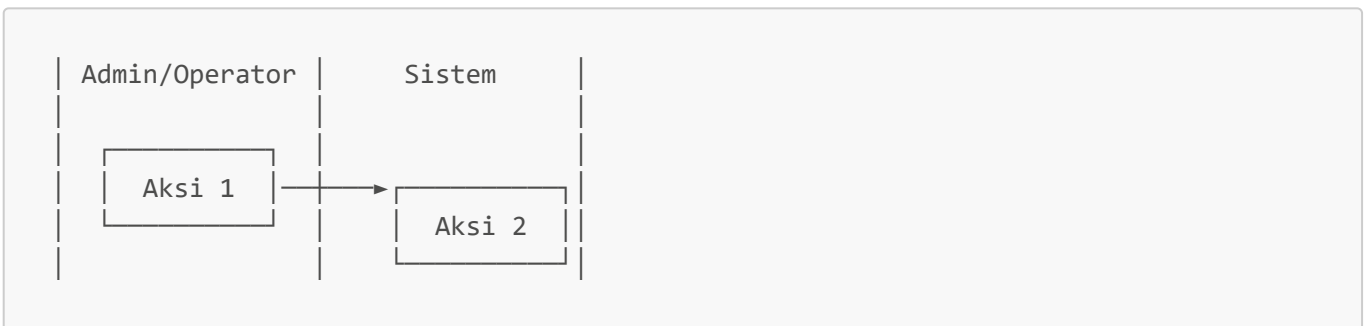
Arti: Memecah proses jadi **PARALEL** (berjalan bersamaan).

7. JOIN NODE (Garis Tebal Horizontal)



Arti: Menunggu semua proses paralel selesai baru lanjut.

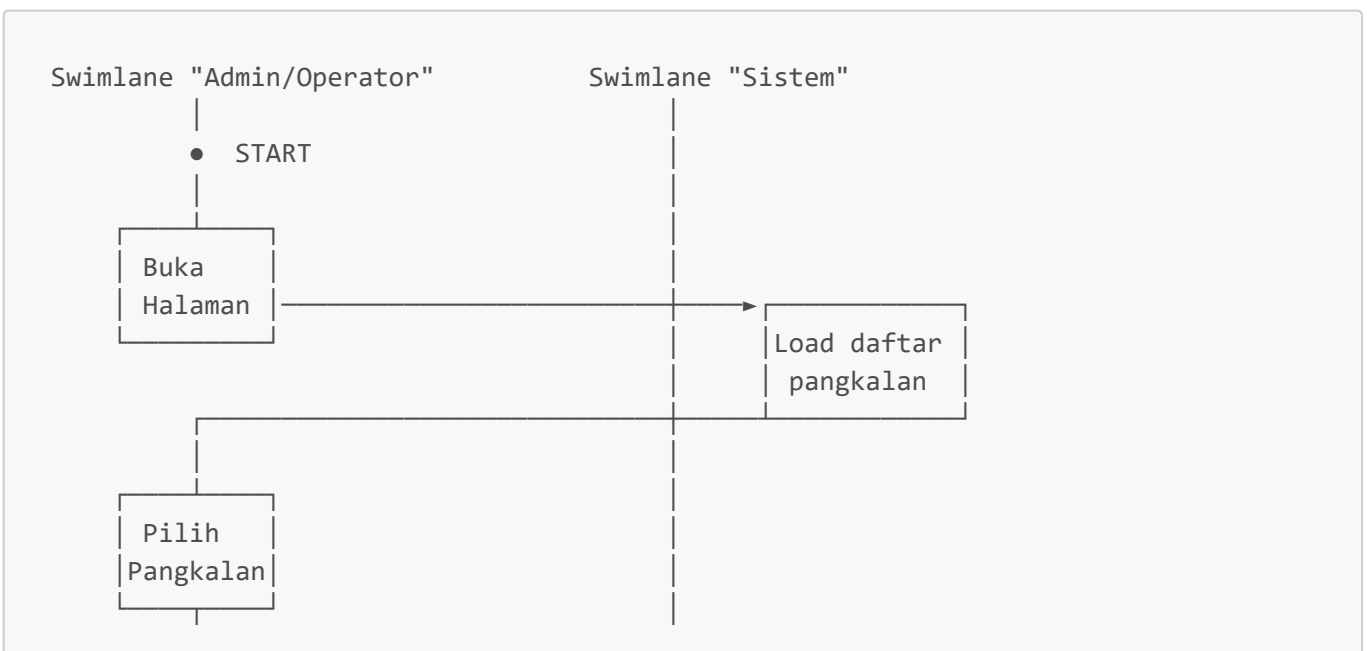
8. SWIMLANE (Kolom Vertikal)

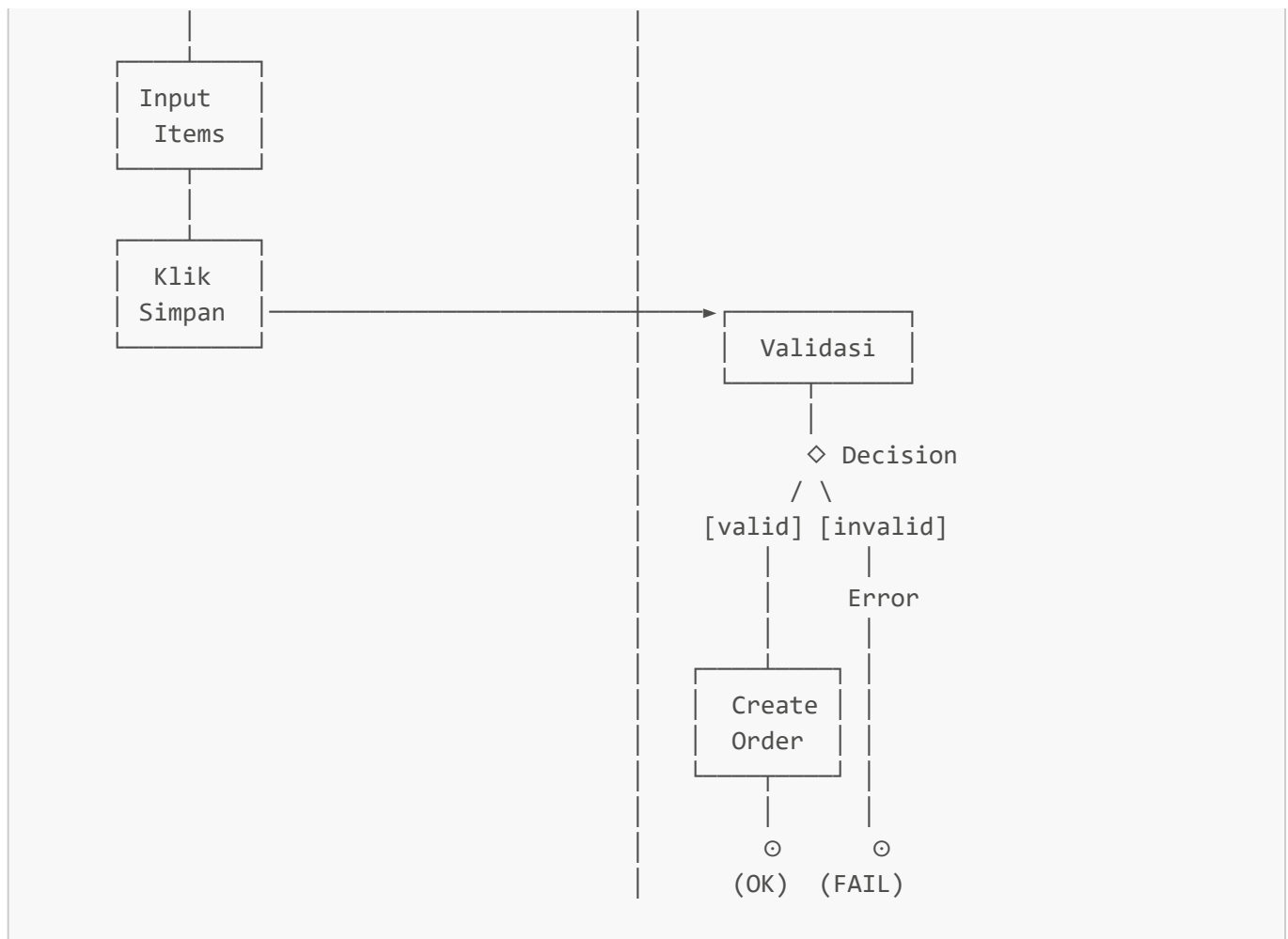


Arti: Memisahkan siapa yang melakukan aksi. Kiri = user, kanan = sistem.

Contoh Activity Diagram di SIM4LON

AD-02: Buat Pesanan





? Pertanyaan yang Mungkin Ditanya Dosen (Activity Diagram)

Q: Kenapa pakai swimlane?

A: Untuk **memperjelas responsibility** - siapa yang melakukan apa. Di SIM4LON ada 2 swimlane: User (Admin/Operator) dan Sistem.

Q: Apa bedanya fork/join dengan decision/merge?

A:

- Fork/Join = **PARALEL** (semua jalur jalan bersamaan)
- Decision/Merge = **PILIHAN** (hanya 1 jalur yang dipilih)

Q: Di AD-03 Update Status, kenapa ada OPT fragment?

A: OPT = **Optional fragment**. Artinya proses itu hanya dilakukan jika kondisi terpenuhi. Contoh: auto-sync stok hanya dilakukan **jika** status berubah ke SELESAI.

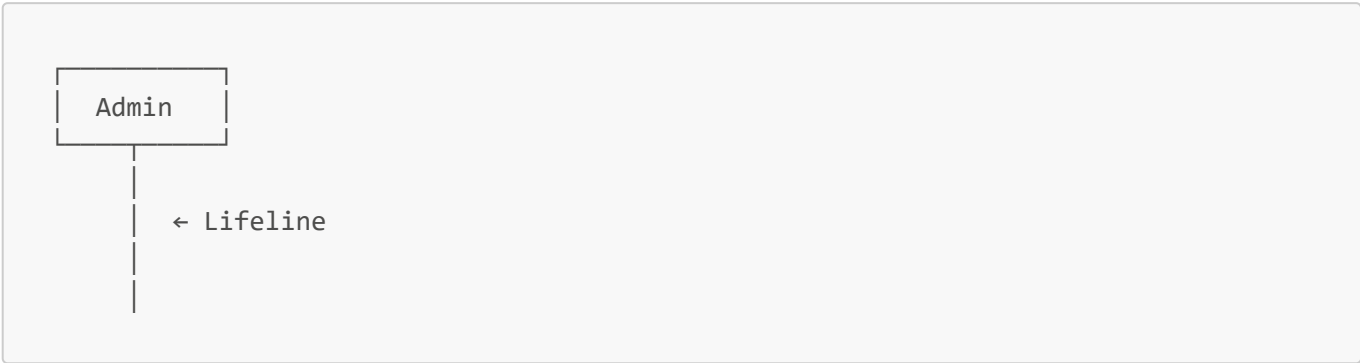
◇ BAGIAN 4: SEQUENCE DIAGRAM

Fungsi Sequence Diagram

Sequence Diagram menggambarkan **INTERAKSI antar komponen** secara kronologis. Fokus ke **urutan pesan** yang dikirim dari satu komponen ke komponen lain.

Simbol-Simbol Sequence Diagram

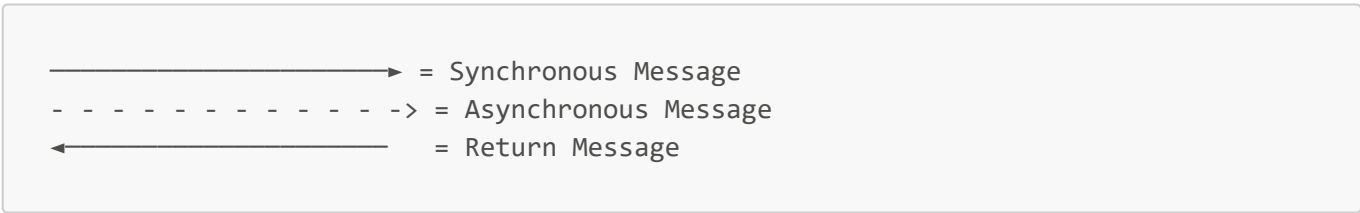
1. PARTICIPANT / LIFELINE (Kotak + Garis Vertikal)



Jenis Participant di SIM4LON:

Stereotype	Simbol	Arti
actor		User/aktor manusia
boundary		UI/Halaman Frontend
control		Service/Logic Backend
entity		Object model
database		Database table

2. MESSAGE (Panah)

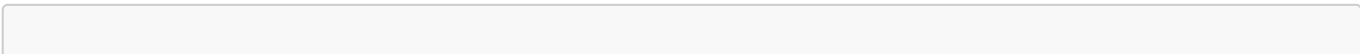


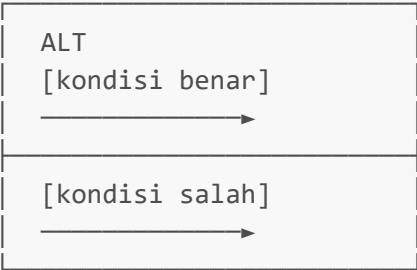
3. ACTIVATION BOX (Kotak Tipis di Lifeline)



4. COMBINED FRAGMENTS

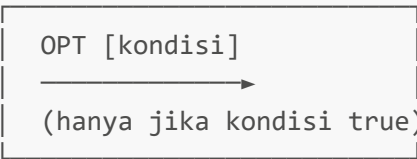
ALT (Alternative) - IF-ELSE





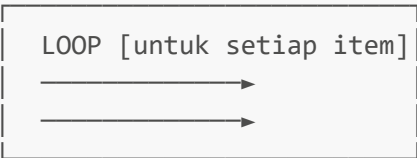
Contoh SIM4LON: Cek apakah user valid atau tidak saat login.

OPT (Optional) - IF saja



Contoh SIM4LON: Auto-sync stok hanya jika status = SELESAI.

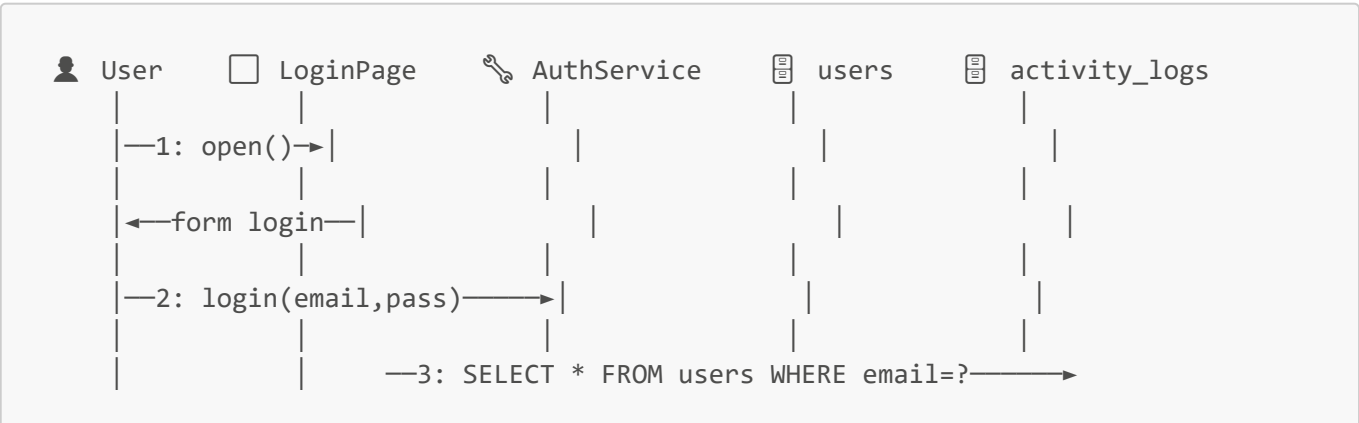
LOOP - Perulangan

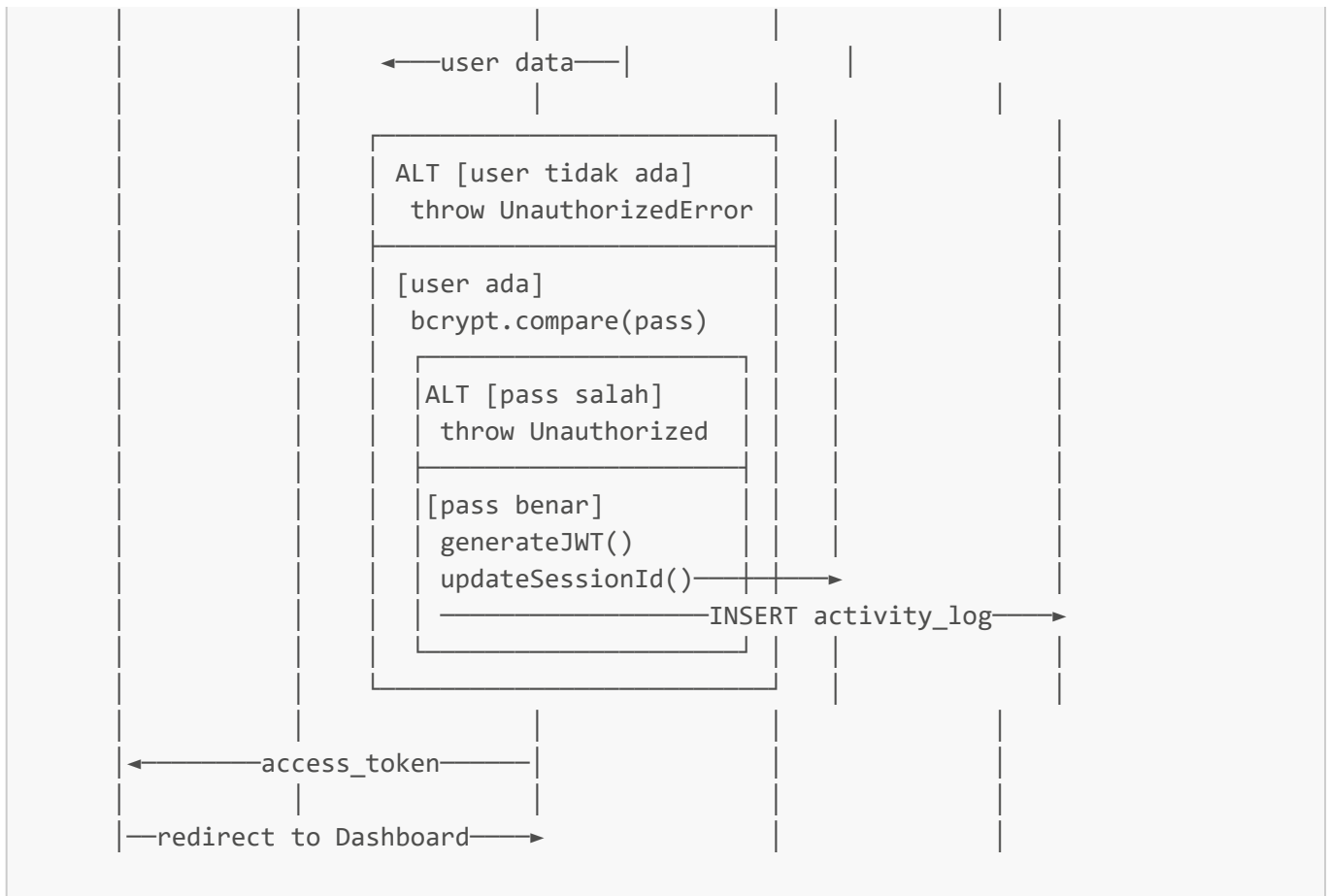


Contoh SIM4LON: Proses setiap item dalam order.

Contoh Sequence Diagram di SIM4LON

SD-01: Login





? Pertanyaan yang Mungkin Ditanya Dosen (Sequence Diagram)

Q: Apa itu stereotype boundary, control, entity?

A: Ini dari pola **BCE (Boundary-Control-Entity)**:

- **Boundary** = UI/interface yang berinteraksi dengan user
- **Control** = Logic/service yang memproses data
- **Entity** = Data/model yang disimpan

Q: Kenapa pakai ALT fragment?

A: Untuk menunjukkan **kondisional/percabangan**. Contoh: jika password benar → lanjut, jika salah → error.

Q: Apa beda synchronous dan asynchronous message?

A:

- Synchronous (garis penuh) = Pengirim **menunggu** balasan
- Asynchronous (garis putus) = Pengirim **tidak menunggu**, langsung lanjut

Q: Apa itu single-session login di SD-01?

A: Setiap login, sistem generate **session_id baru** dan simpan di database. Jika user login di device lain, session_id lama jadi invalid → user lama otomatis logout. Ini untuk **keamanan**.

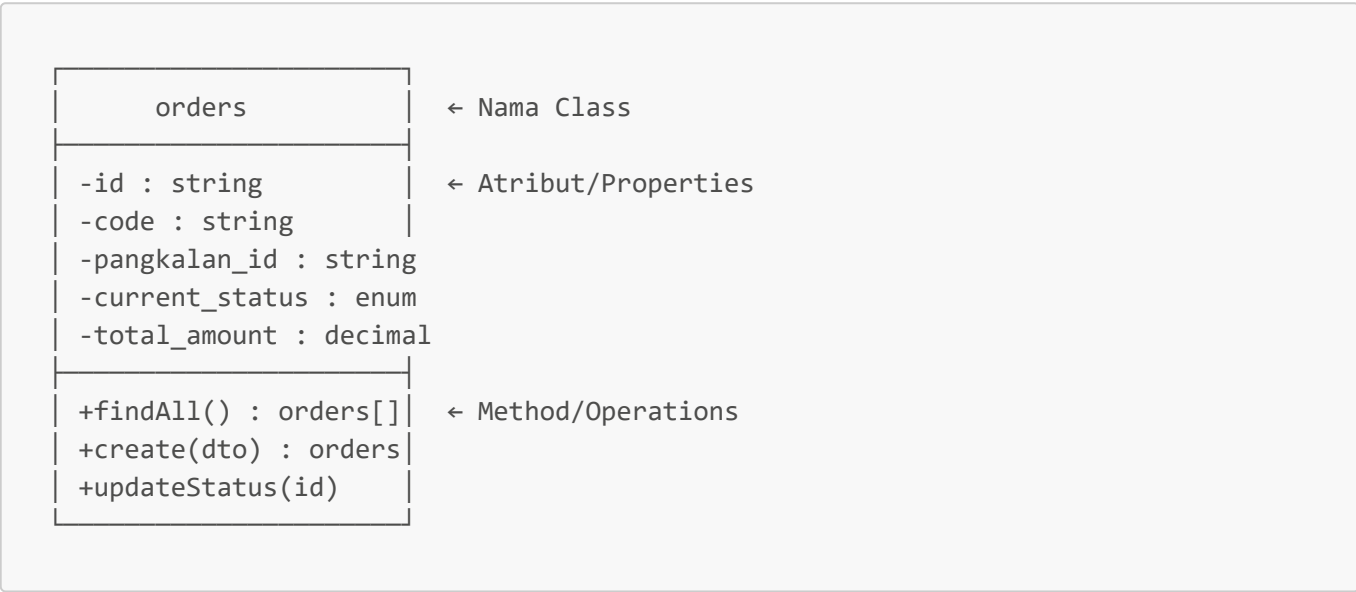
 BAGIAN 5: CLASS DIAGRAM

Fungsi Class Diagram

Class Diagram menggambarkan **STRUKTUR OBJECT-ORIENTED** dari sistem. Menunjukkan class apa saja, atribut dan method-nya, serta relasi antar class.

Simbol-Simbol Class Diagram

1. CLASS (Kotak 3 Bagian)



2. VISIBILITY (Simbol Akses)

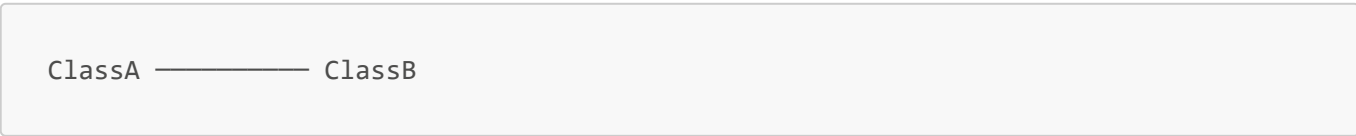
Simbol	Nama	Arti
-	Private	Hanya bisa diakses dari dalam class sendiri
+	Public	Bisa diakses dari mana saja
#	Protected	Bisa diakses dari class sendiri dan turunannya
~	Package	Bisa diakses dari package yang sama

Di SIM4LON:

- Atribut pakai - (private) → enkapsulasi
- Method pakai + (public) → bisa dipanggil dari luar

3. RELATIONSHIPS

Association (Garis Biasa)



Arti: A dan B saling berhubungan/tahu satu sama lain.

Aggregation (Diamond Kosong)

```
ClassA ◇----- ClassB
```

Arti: A "punya" B, tapi B bisa exist sendiri tanpa A.

Contoh SIM4LON:

```
Agen ◇----- Pangkalan
```

Agan punya banyak pangkalan, tapi kalau agan dihapus, data pangkalan **tetap ada**.

Composition (Diamond Penuh)

```
ClassA ◆----- ClassB
```

Arti: A "punya" B, dan B **tidak bisa exist tanpa A**.

Contoh SIM4LON:

```
Orders ◆----- OrderItems
```

Order punya items, dan kalau order dihapus, **items juga ikut terhapus**.

Dependency (Garis Putus-putus dengan Panah)

```
ClassA - - - - -> ClassB
```

Arti: A menggunakan B, tapi tidak "memiliki" B.

Contoh SIM4LON:

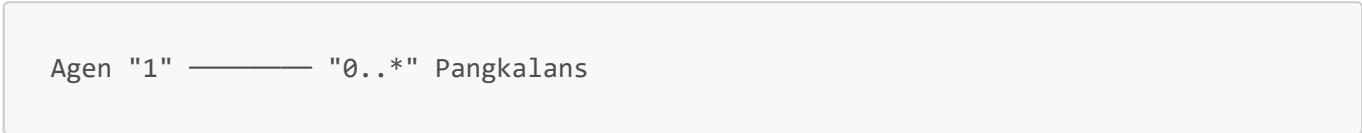
```
orders - - - - -> status_pesanan (enum)
```

Order **menggunakan** enum status_pesanan.

4. MULTIPLICITY (Angka di Ujung Garis)

Notasi	Arti
1	Tepat 1
0..1	Nol atau 1 (opsional)
* atau 0..*	Nol atau banyak
1..*	Minimal 1 atau banyak
n	Tepat n

Contoh SIM4LON:



Artinya: 1 Agen bisa punya **0 atau banyak** Pangkalan.

Package di SIM4LON Class Diagram

Package	Isi	Warna
Enumerations	9 enum (user_role, status_pesanan, dll)	Kuning
Master Data	users, agen, pangkalans, drivers, lpg_products	Hijau
Order Management	orders, order_items, timeline_tracks, invoices	Biru
Payment	order_payment_details, payment_records	Oranye
Stock Management	stock_histories, penerimaan_stok, penyaluran_harian	Pink
Pangkalan SAAS	consumers, consumer_orders, pangkalan_stocks, expenses	Ungu
Audit & Logging	activity_logs	Abu

? Pertanyaan yang Mungkin Ditanya Dosen (Class Diagram)

Q: Kenapa pakai composition untuk orders-order_items?

A: Karena order_item **tidak bisa exist tanpa order**. Jika order dihapus, otomatis item-nya juga terhapus. Ini **strong ownership**.

Q: Kenapa pangkalan_id ada di banyak class?

A: Untuk **multi-tenant isolation**. Setiap data yang berhubungan dengan pangkalan harus punya FK ke pangkalan_id agar bisa di-filter per pangkalan.

Q: Apa itu aggregate root?

A: Class utama yang "mengontrol" class-class terkait. Di SIM4LON, **Orders adalah aggregate root** yang mengontrol OrderItems, TimelineTracks, dan Payment.

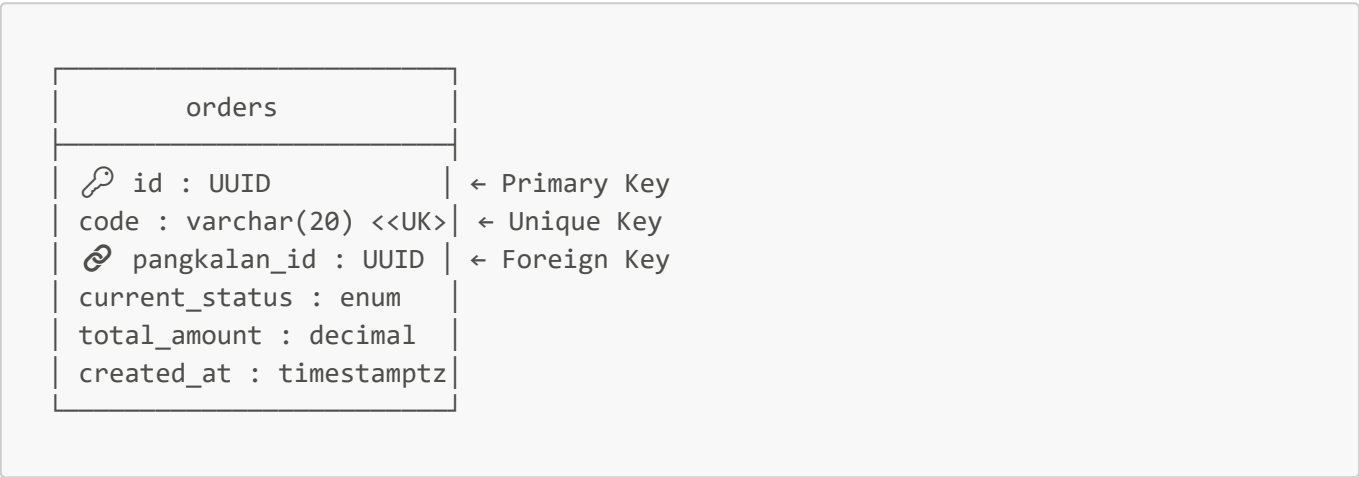
BAGIAN 6: ERD (ENTITY RELATIONSHIP DIAGRAM)

Fungsi ERD

ERD menggambarkan **STRUKTUR DATABASE** - table apa saja, kolom-kolomnya, dan relasi antar table.

Simbol-Simbol ERD

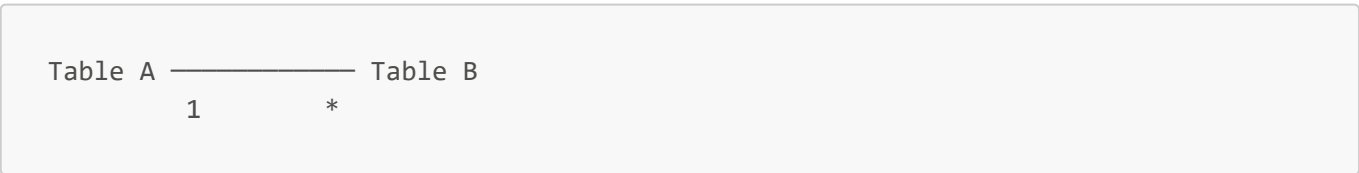
1. ENTITY (Table)



2. KEY TYPES

Simbol	Nama	Arti
 PK	Primary Key	ID unik untuk setiap record
 FK	Foreign Key	Reference ke table lain
<>	Unique Key	Nilai harus unik, tapi bukan PK

3. RELATIONSHIPS (Cardinality)



Notasi	Arti
1 — 1	One-to-One
1 — *	One-to-Many
* — *	Many-to-Many (perlu junction table)

Design Decisions di ERD SIM4LON

1. UUID sebagai Primary Key

```
id : UUID -- bukan INTEGER AUTO_INCREMENT
```

Alasan:

- **Security:** ID tidak bisa ditebak (tidak sequential)
- **Distributed:** Bisa generate ID di client tanpa collision
- **Merging:** Mudah merge data dari berbagai sumber

2. Soft Delete

```
deleted_at : timestampz NULL
```

Alasan:

- Data sensitif tidak dihapus permanen
- Bisa di-restore jika diperlukan
- Audit trail tetap lengkap

3. Timestamps

```
created_at : timestampz  
updated_at : timestampz
```

Alasan: Untuk audit trail - kapan data dibuat dan terakhir diubah.

4. UNIQUE Constraint di pangkalan_stocks

```
UNIQUE (pangkalan_id, lpg_type)
```

Alasan: Enable **UPSERT** untuk auto-sync stok. Jika kombinasi sudah ada → UPDATE, jika belum → INSERT.

? Pertanyaan yang Mungkin Ditanya Dosen (ERD)

Q: Kenapa pakai UUID, bukan auto-increment?

A: Untuk **security** (tidak bisa ditebak) dan **distributed readiness** (bisa generate di client).

Q: Apa itu soft delete dan kenapa pakai itu?

A: Soft delete adalah menandai data sebagai "dihapus" (set deleted_at = now) tanpa benar-benar menghapus. Alasan: **data recovery** dan **audit trail**.

Q: Database normalized sampai level berapa?

A: **Third Normal Form (3NF)**. Tidak ada redundansi data, semua non-key column depend on primary key.

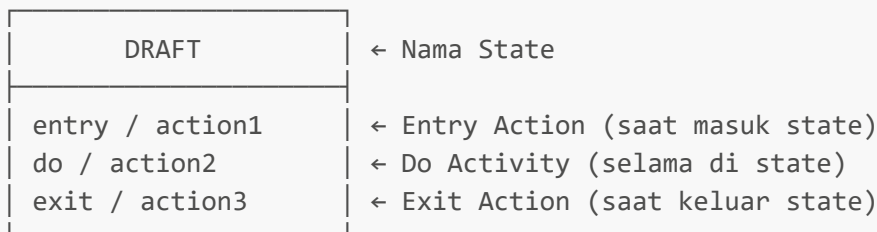
BAGIAN 7: STATE MACHINE DIAGRAM

Fungsi State Machine Diagram

State Machine Diagram menggambarkan **LIFECYCLE OBJECT** - state apa saja yang bisa terjadi dan bagaimana transisi antar state.

Simbol-Simbol State Machine Diagram

1. STATE (Kotak dengan Sudut Lengkung atau Bulat)



2. INITIAL STATE (Bulatan Hitam Penuh)

● = Titik awal

3. FINAL STATE (Bulatan dengan Lingkaran)

⊙ = Titik akhir

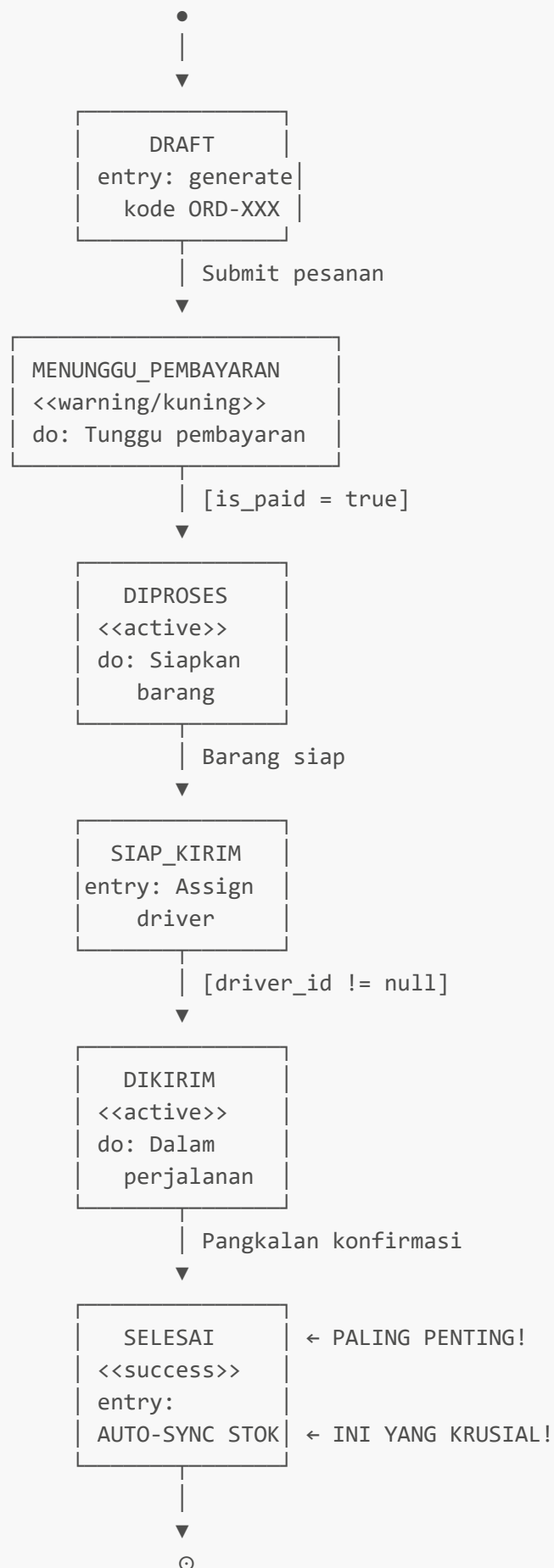
4. TRANSITION (Panah dengan Label)

State A —[trigger/guard]→ State B

- **trigger**: Event yang memicu transisi
- **guard**: Kondisi yang harus terpenuhi [dalam kurung kotak]

State Machine di SIM4LON

SM-01: Order Status (PALING PENTING!)



* BATAL bisa dari state manapun kecuali SELESAI

? Pertanyaan yang Mungkin Ditanya Dosen (State Machine)

Q: Apa itu entry action?

A: Aksi yang **otomatis dijalankan** saat object masuk ke state tersebut. Contoh: saat masuk SELESAI, otomatis jalankan auto-sync stok.

Q: Kenapa BATAL tidak bisa dari SELESAI?

A: Karena di state SELESAI, **stok sudah di-sync ke pangkalan**. Jika dibatalkan, akan terjadi inkonsistensi data stok.

Q: State Machine ini hanya dokumentasi atau benar-benar diimplementasikan?

A: **Benar-benar diimplementasikan** sebagai validation logic di backend. Jika ada transisi yang tidak valid, sistem akan **reject** dengan error.

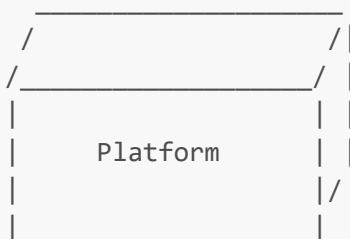
🌀 BAGIAN 8: DEPLOYMENT DIAGRAM

Fungsi Deployment Diagram

Deployment Diagram menggambarkan **ARSITEKTUR FISIK** sistem - hardware/platform apa yang dipakai dan bagaimana komponen software di-deploy.

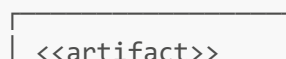
Simbol-Simbol Deployment Diagram

1. NODE (Kotak 3D)



Arti: Hardware atau platform tempat software di-deploy.

2. ARTIFACT (Kotak Biasa dalam Node)



NestJS 11

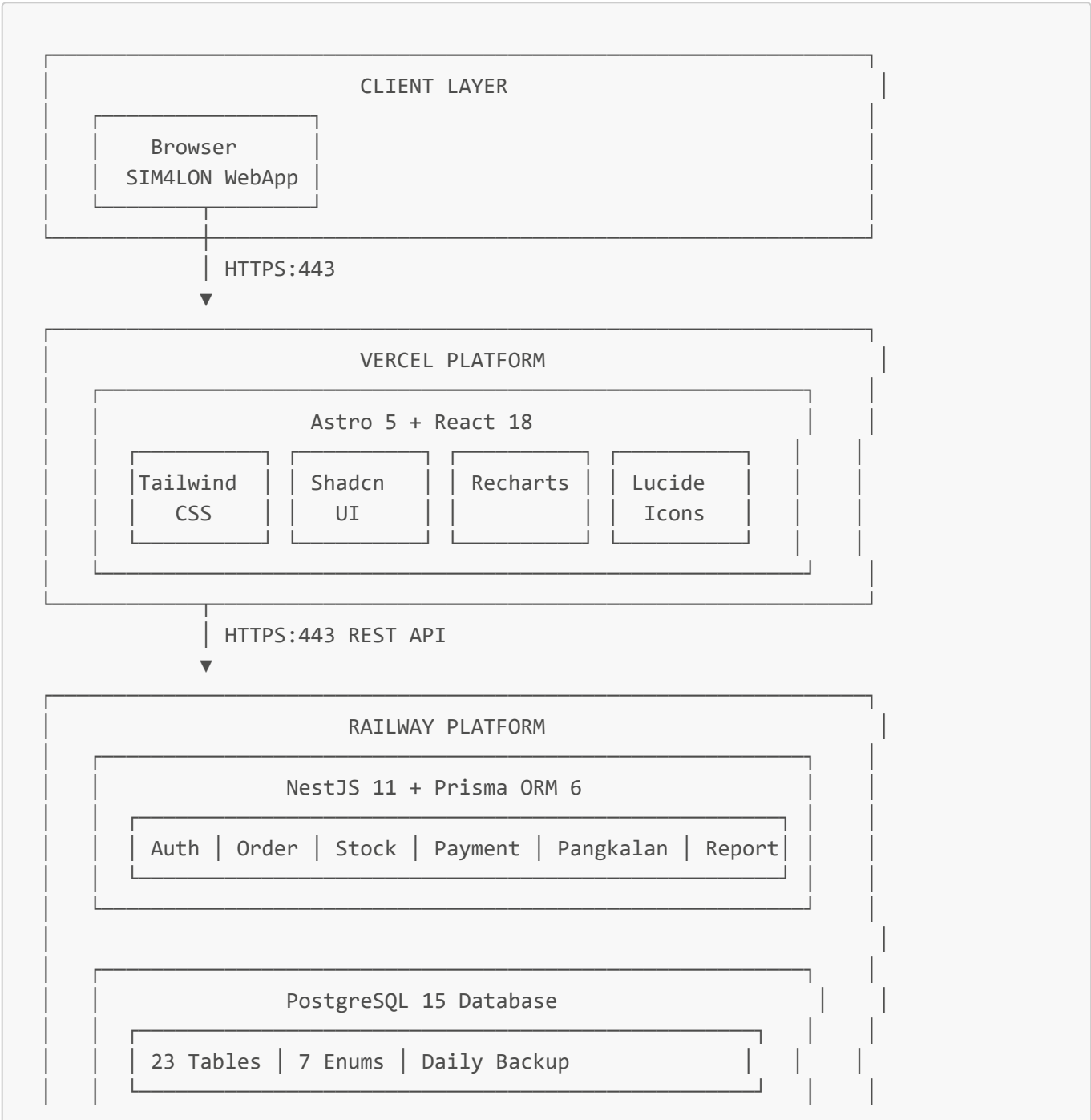
Arti: Software/komponen yang di-deploy di node tersebut.

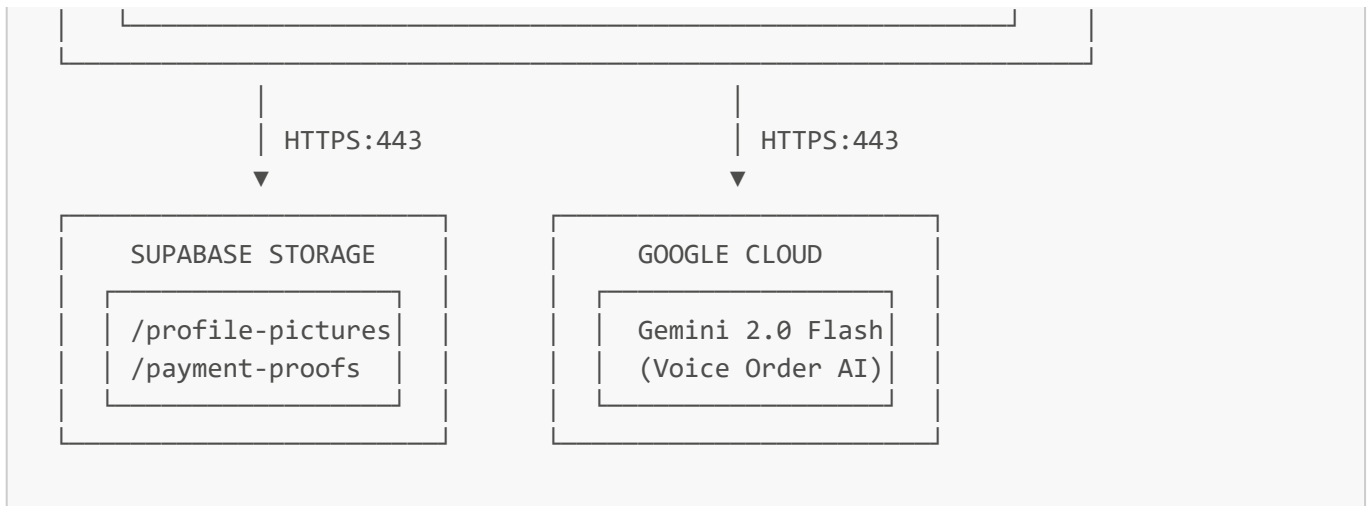
3. CONNECTION (Garis dengan Label Protocol)

Node A —HTTPS:443—> Node B

Arti: Komunikasi antar node dengan protocol tertentu.

Deployment SIM4LON





? Pertanyaan yang Mungkin Ditanya Dosen (Deployment)

Q: Kenapa deploy di Vercel dan Railway?

A:

- **Vercel** optimal untuk frontend React/Next.js dengan CDN global
- **Railway** mudah untuk deploy NestJS dengan PostgreSQL included
- Keduanya punya **free tier** yang generous

Q: Bagaimana security di production?

A:

- Semua komunikasi pakai **HTTPS (port 443)**
- JWT untuk authentication
- Role-based access control
- Single-session login untuk prevent session hijacking

Q: Kenapa pakai Supabase untuk storage?

A: Supabase menyediakan S3-compatible storage dengan **Row Level Security (RLS)** - user hanya bisa akses file mereka sendiri.

🎯 BAGIAN 9: TIPS MENJAWAB PERTANYAAN DOSEN

Formula Menjawab

1. **Pahami pertanyaan dulu** - jangan langsung jawab
2. **Jawab singkat dan jelas** - to the point
3. **Kaitkan dengan diagram** - "Seperti yang terlihat di diagram X..."
4. **Berikan contoh konkret** - pakai contoh dari SIM4LON
5. **Akui jika tidak tahu** - "Untuk detail itu perlu riset lebih lanjut, tapi secara prinsip..."

Pertanyaan Umum dan Jawaban

Q: "Kenapa pakai UML ini, bukan yang lain?"

A: Saya memilih UML ini berdasarkan **kebutuhan dokumentasi**:

- **Use Case**: Untuk menunjukkan fitur dari sudut pandang user
- **Activity**: Untuk menjelaskan alur bisnis step-by-step
- **Sequence**: Untuk menunjukkan interaksi teknis antar komponen
- **Class + ERD**: Untuk menjelaskan struktur data
- **State Machine**: Untuk menjelaskan lifecycle object kunci (order)
- **Deployment**: Untuk menjelaskan arsitektur production

Q: "Bagaimana cara Anda validasi bahwa diagram sudah benar?"

A: Saya melakukan **cross-validation**:

1. Class Diagram ↔ ERD → Harus consistent
2. Sequence Diagram ↔ Kode → Method yang dipanggil harus ada di class
3. State Machine → Implemented di backend sebagai validation logic
4. Use Case → Semua fitur bisa di-demo

Q: "Apa yang paling challenging dalam membuat UML ini?"

A: Bagian paling challenging adalah **State Machine Order Status** karena:

- Harus memikirkan semua kemungkinan transisi
- Entry action (auto-sync) harus tepat timing-nya
- Validasi transisi harus benar-benar di-implement di code

FINAL CHECKLIST SEBELUM SIDANG

- ☐ Bisa jelaskan **fungsi setiap diagram**
- ☐ Bisa jelaskan **simbol-simbol utama**
- ☐ Bisa **trace flow** di Activity Diagram
- ☐ Bisa jelaskan **relationships** di Class Diagram dan ERD
- ☐ Bisa jelaskan **state transitions** di State Machine
- ☐ Bisa jelaskan **arsitektur deployment**
- ☐ Bisa kaitkan diagram dengan **kode/implementasi**
- ☐ Bisa jawab "kenapa pakai X, bukan Y?"

Good luck untuk sidangnya! Bismillah, Grade A!  